

NMIGOD 参数配置深度解析

邻域互信息图卷积异常检测算法

2026-07-23

目录

1	算法概述	3
2	参数体系总览	3
3	第一层：用户可配置超参数 (7 个)	3
3.1	lambda_param (λ): 邻域半径系数	3
3.2	mi_threshold (τ): 互信息稀疏化阈值	4
3.3	hidden1, hidden2: GCN 隐藏层维度	4
3.4	epochs, lr: 训练超参数	4
3.5	labeled_ratio: 标注比例	5
4	第二层：固定配置参数 (4 个)	5
4.1	pos_weight 动态计算机制	5
5	第三层：算法内部计算参数 (5 个)	5
5.1	参考半径 σ_a	5
5.2	划分信息熵 NE_a (公式 10)	6
5.3	全局密度 ρ_a	6
5.4	自适应半径 ε_a (公式 12)	6
5.5	邻域互信息矩阵 M	6
6	第四层：消融实验控制开关 (2 个)	7
6.1	纯 NMI 模式 (use_gcn=False)	7
7	参数敏感性分析与推荐配置	7
7.1	网格搜索结果	7
7.2	参数影响链路图	8

8	完整命令行使用示例	8
8.1	默认参数运行	8
8.2	使用推荐配置	8
8.3	多数据集批量运行	9
8.4	Python API 调用 (含消融实验)	9
9	参数调优实战指南	10
9.1	调参优先级	10
9.2	不同数据类型的调参建议	10
9.3	诊断输出解读	10
10	总结	11

1 算法概述

NMIGOD (Neighborhood Mutual Information and Graph Convolutional Network based Outlier Detection) 是一种面向混合属性数据的半监督异常检测算法。其核心创新在于：

- 自适应邻域半径**：基于连通分量信息熵动态计算每个数值属性的邻域半径 $\varepsilon_a = \sigma_a / (1 + \rho_a)$
- 邻域互信息图**：用邻域互信息 (NMI) 度量对象间的结构相似性，构建稀疏化的信息图
- GCN 半监督分类**：在图结构上进行两层图卷积网络的二分类，以异常概率作为异常分数

2 参数体系总览

NMIGOD 的参数体系分为四个层次：

表 1: NMIGOD 参数体系四层架构

层次	参数类别	参数数量
第一层	暴露给用户的超参数 (CLI/构造函数)	7 个
第二层	固定配置参数 (代码内硬编码)	4 个
第三层	算法内部计算参数 (数据驱动)	5 个
第四层	消融实验控制开关	2 个

3 第一层：用户可配置超参数 (7 个)

3.1 lambda_param (λ)：邻域半径系数

表 2: lambda_param 参数详情

属性	说明
CLI 参数	<code>--lambda-param</code>
默认值	1.0
推荐值	0.5 (经网格搜索验证)
类型	float
作用机制	控制邻域半径的缩放系数。 λ 影响参考半径 σ_a 的构建，进而间接影响自适应半径 $\varepsilon_a = \sigma_a / (1 + \rho_a)$ 。
调参建议	$\lambda \in [0.5, 1.5]$ 。较小的 λ 产生更小、更具判别力的邻域；较大的 λ 使邻域更宽松，可能引入噪声连接。
影响范围	直接影响邻域图中的边数，间接影响 NMI 矩阵的稀疏性和 GCN 的消息传递范围

3.2 mi_threshold (τ): 互信息稀疏化阈值

表 3: mi_threshold 参数详情

属性	说明
CLI 参数	<code>--mi-threshold</code>
默认值	0.05
推荐值	0.03 (经网格搜索验证)
类型	float
作用机制	对归一化后的 NMI 矩阵 M 进行硬阈值过滤: $M_{ij} = 0$ 若 $M_{ij} < \tau$ 。
调参建议	$\tau \in [0.01, 0.1]$ 。较低的 τ 保留更多的结构连接; 较高的 τ 使图更稀疏, 降低计算开销但可能丢失有用信息。
影响范围	决定 GCN 输入图的稀疏度和拓扑结构。过于稀疏的图会导致 GCN 欠拟合; 过于稠密的图会引入噪声。

3.3 hidden1, hidden2: GCN 隐藏层维度

表 4: GCN 架构参数详情

属性	说明
CLI 参数	<code>--hidden1 / --hidden2</code>
默认值	hidden1=128, hidden2=64
类型	int
作用机制	GCN 采用两层结构: 第一层 $X \rightarrow h_{128}$ (ReLU + Dropout), 第二层 $h_{128} \rightarrow h_{64}$ (ReLU), 最终线性层 $h_{64} \rightarrow 1$ (logits)。
调参建议	根据输入特征维度调整。特征维度高时可适当增大; 小数据集建议减小以避免过拟合。
模型结构	$\underbrace{X}_{d_{in}} \xrightarrow{\text{GC}_1} \underbrace{h_1}_{128} \xrightarrow[\text{Dropout}]{\text{ReLU}} \underbrace{h_1}_{128} \xrightarrow{\text{GC}_2} \underbrace{h_2}_{64} \xrightarrow{\text{ReLU}} \underbrace{h_2}_{64} \xrightarrow{\text{Linear}} \underbrace{\hat{y}}_1$

3.4 epochs, lr: 训练超参数

表 5: 训练超参数详情

属性	epochs	lr
CLI 参数	<code>--epochs</code>	<code>--lr</code>
默认值	200	0.01
类型	int	float
作用	GCN 训练迭代轮数	Adam 优化器学习率
推荐范围	[100, 500]	[0.001, 0.01]
说明	配合早停策略使用更佳	综合报告中使用 0.001

3.5 labeled_ratio: 标注比例

表 6: labeled_ratio 参数详情

属性	说明
构造函数参数	<code>labeled_ratio</code>
默认值	0.2 (20%)
类型	<code>float</code>
CLI 暴露	否 (仅 Python API)
作用	半监督学习中标注样本的比例。标注集中 75% 用于训练, 25% 用于验证。
划分方式	全量数据的 20% 作为标注集 (其中 75% 训练, 25% 验证), 80% 作为无标签评估集。
说明	使用分层采样 (stratify) 保持训练/验证/评估集中的异常比例一致。

4 第二层：固定配置参数 (4 个)

表 7: 代码内硬编码的固定参数

参数	值	位置	说明
<code>dropout</code>	0.5	<code>GCNClassifier.__init__</code>	GCN 层间 Dropout 率, 防止过拟合
<code>weight_decay</code>	5×10^{-4}	<code>train_model</code>	Adam 优化器 L2 正则化系数
<code>random_state</code>	42	构造函数	全局随机种子, 保证可复现性
<code>pos_weight_cap</code>	100.0	<code>train_model</code>	BCE 损失中正类权重的上界

4.1 pos_weight 动态计算机制

BCEWithLogitsLoss 的 `pos_weight` 根据训练集正负样本比自动计算:

$$\text{pos_weight} = \min \left(\frac{N_{neg}^{train}}{N_{pos}^{train}}, 100.0 \right)$$

这确保模型在面对极度不平衡的异常检测数据时能够有效学习正类（异常）样本。

5 第三层：算法内部计算参数 (5 个)

5.1 参考半径 σ_a

- 定义: $\sigma_a = \text{std}(X_a^{norm})$, 即数值属性 a 在 $[0, 1]$ 归一化后的标准差
- 意义: 反映属性 a 的离散程度, 作为邻域半径的基准尺度
- 边界处理: 若 $\sigma_a = 0$, 则设为 10^{-6} 避免除零

5.2 划分信息熵 NE_a (公式 10)

- 定义: $NE_a = -\sum_i \frac{|C_i|}{|U|} \log_2 \frac{|C_i|}{|U|}$
- 计算方法: 对属性 a 的值排序后, 以 σ_a 为间隔阈值切分连通分量 $\{C_i\}$
- 连通分量判定: 排序序列中相邻值之差 $> \sigma_a$ 即为断点
- 意义: 度量属性 a 上数据分布的聚集程度。 NE_a 越高说明对象分布越均匀 (密度越低), NE_a 越低说明存在明显的聚集结构 (密度越高)

5.3 全局密度 ρ_a

- 定义: $\rho_a = 1 - \frac{NE_a}{\log_2 |U|}$
- 值域: 经裁剪后 $\rho_a \in [0, 1]$
- 解释:
 - $\rho_a \rightarrow 1$: 属性 a 上数据高度聚集 (连通分量少, 信息熵低)
 - $\rho_a \rightarrow 0$: 属性 a 上数据均匀分布 (连通分量多, 信息熵高)
- 作用: 作为自适应半径的分母调节因子

5.4 自适应半径 ε_a (公式 12)

- 定义: $\varepsilon_a = \frac{\sigma_a}{1 + \rho_a}$
- 核心直觉:
 - 密集属性的 ρ_a 大, ε_a 缩小 $\rightarrow$ 在密集区域需要更精细的分辨
 - 稀疏属性的 ρ_a 小, ε_a 接近 σ_a $\rightarrow$ 在稀疏区域放宽邻域范围
- 值域: $\varepsilon_a \in [\sigma_a/2, \sigma_a]$
- 双向规则: 最终邻域判定使用 $|x_i^a - x_j^a| \leq \min(\varepsilon_a(x_i), \varepsilon_a(x_j))$

5.5 邻域互信息矩阵 M

- 构建流程:
 1. 逐属性构建邻域掩码 N_{mask} : 数值属性用 ε 阈值, 类别属性用相等判断
 2. 计算邻域交集: $intersection_{ij} = |N(i) \cap N(j)|$
 3. 计算 NMI: $I_{ij} = \frac{|N(i) \cap N(j)|}{N} \cdot \log_2 \left(\frac{N \cdot |N(i) \cap N(j)|}{|N(i)| \cdot |N(j)|} \right)$
 4. 归一化: $M = I / \max(I_{off-diag})$
 5. 稀疏化: $M_{ij} = 0$ if $M_{ij} < \tau$
 6. 对称归一化: $\hat{M} = D^{-1/2} M D^{-1/2}$
- 物理意义: M_{ij} 度量对象 x_i 和 x_j 在整个属性空间中的邻域结构相似性

6 第四层：消融实验控制开关 (2 个)

表 8: 消融实验控制参数

参数	默认值	关闭效果	实验目的
use_adaptive_radius	True	$\varepsilon_a = \sigma_a$ (无自适应)	验证自适应半径对性能的贡献
use_gcn	True	跳过 GCN, 使用纯 NMI 分数	验证 GCN 图学习对性能的贡献

6.1 纯 NMI 模式 (use_gcn=False)

当关闭 GCN 时，异常分数直接从 NMI 矩阵导出：

$$score(x_i) = 1 - \frac{1}{|U| - 1} \sum_{j \neq i} M_{ij}$$

即对象 x_i 与所有其他对象的平均 NMI 越低，越可能是异常。

7 参数敏感性分析与推荐配置

7.1 网格搜索结果

综合报告中对 (λ, τ) 进行了 9 组网格搜索 ($\lambda \in \{0.5, 1.0, 1.5\}$, $\tau \in \{0.03, 0.05, 0.1\}$)，验证数据集为 iris, wine, glass, diabetes。

表 9: 推荐配置 vs 默认配置

参数	默认值	推荐值	调整理由
lambda_param (λ)	1.0	0.5	更小邻域提高判别力
mi_threshold (τ)	0.05	0.03	保留更多结构连接
hidden1	128	128	保持不变
hidden2	64	64	保持不变
epochs	200	200	保持不变
lr	0.01	0.001	综合报告中使用的学习率
labeled_ratio	0.2	0.2	20% 标注为常规半监督设置

7.2 参数影响链路图

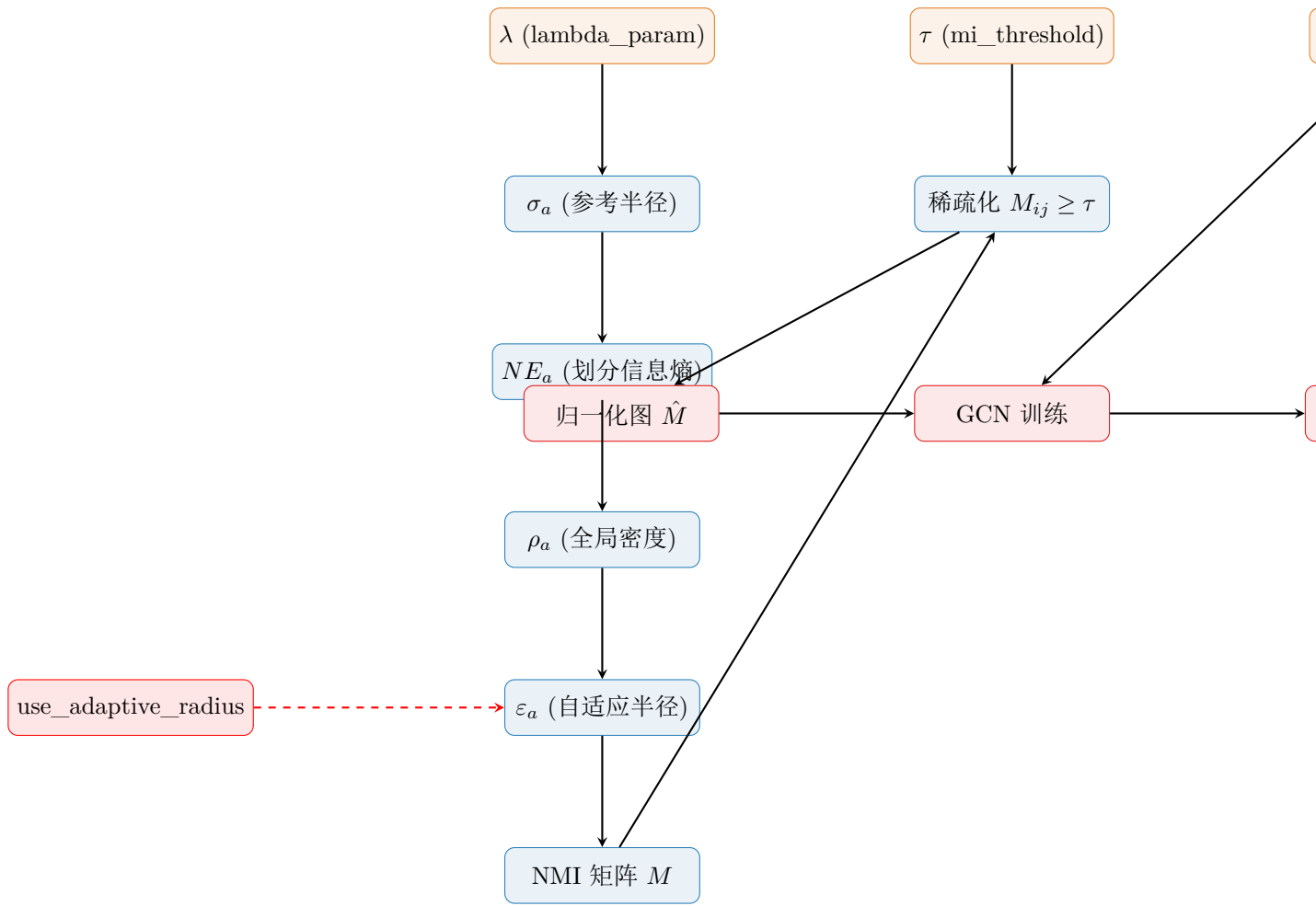


图 1: NMIGOD 参数影响链路：从用户超参数到最终异常分数

8 完整命令行使用示例

8.1 默认参数运行

```
python detector.py --datasets data.csv --target label --anomaly anomaly
```

8.2 使用推荐配置

```
python detector.py \
  --datasets data.csv \
  --target label \
  --anomaly anomaly \
  --lambda-param 0.5 \
  --mi-threshold 0.03 \
  --lr 0.001 \
  --epochs 200
```


8.3 多数据集批量运行

```
python detector.py \  
    --datasets data1.csv,data2.csv,data3.csv \  
    --target label \  
    --anomaly anomaly \  
    --lambda-param 0.5 \  
    --mi-threshold 0.03 \  
    --lr 0.001 \  
    --output ./results
```

8.4 Python API 调用 (含消融实验)

```
from detector import AnomalyDetectionFramework  
  
# 完整 NMIGOD (推荐配置)  
detector = AnomalyDetectionFramework(  
    lambda_param=0.5,  
    mi_threshold=0.03,  
    hidden1=128, hidden2=64,  
    epochs=200, lr=0.001,  
    labeled_ratio=0.2,  
    use_adaptive_radius=True, # 自适应半径开启  
    use_gcn=True             # GCN 分类器开启  
)  
  
# 消融实验 1: 固定半径 (关闭自适应)  
detector_no_ada = AnomalyDetectionFramework(  
    lambda_param=0.5,  
    mi_threshold=0.03,  
    use_adaptive_radius=False, # epsilon_a = sigma_a  
    use_gcn=True  
)  
  
# 消融实验 2: 纯 NMI (关闭 GCN)  
detector_nmi_only = AnomalyDetectionFramework(  
    lambda_param=0.5,  
    mi_threshold=0.03,  
    use_adaptive_radius=True,  
    use_gcn=False # score = 1 - mean(NMI)  
)
```

9 参数调优实战指南

9.1 调参优先级

1. 第一优先级： λ 和 τ ——直接影响图结构质量，对性能影响最大。建议在验证集上网格搜索 $\lambda \in \{0.3, 0.5, 0.8, 1.0, 1.5\}$ 和 $\tau \in \{0.01, 0.03, 0.05, 0.08, 0.1\}$ 。
2. 第二优先级：lr ——GCN 训练收敛的关键。若训练损失震荡或不收敛，尝试降低 lr 至 0.001 或 0.0005。
3. 第三优先级：hidden1, hidden2 ——根据输入特征维度调整。特征数 < 50 时可尝试 (64, 32)；特征数 > 200 时可尝试 (256, 128)。
4. 第四优先级：epochs ——观察损失曲线，一般在 200 轮左右收敛。可配合早停策略。

9.2 不同数据类型的调参建议

表 10: 按数据类型推荐配置

数据类型	λ	τ	说明
纯数值型	0.3–0.5	0.02–0.03	数值属性的 σ_a 更可靠，可用更小的 λ
混合型	0.5–0.8	0.03–0.05	类别属性稀释了距离度量，需适中的半径
纯类别型	0.8–1.5	0.03–0.05	类别属性无距离概念， λ 影响较小
高维数据	0.5–1.0	0.05–0.10	维数灾难下需要更稀疏的图结构

9.3 诊断输出解读

训练过程中的关键诊断信息：

- [*] 构建邻域互信息图 (N=452)...
- [*] 互信息图构建完成，边数(含自环)：15234，非零边比例：7.45%
平均 NE=2.3456，平均 rho=0.6187，平均 eps=0.0432，平均 sigma=0.0712
- 非零边比例：建议在 5%–20% 之间。太低说明图过于稀疏，GCN 消息传递受限；太高说明引入了过多噪声边。
- 平均 ρ ：接近 1 表示数据高度聚集，接近 0 表示数据均匀分散。中等的 ρ (0.4–0.7) 通常对应合理的邻域划分。
- 平均 ε vs σ ： $\varepsilon < \sigma$ 说明自适应机制正在收紧某些属性的邻域半径。

10 总结

表 11: NMIGOD 参数配置速查表

参数	默认	推荐	类型	调参优先级
lambda_param (λ)	1.0	0.5	float	最高
mi_threshold (τ)	0.05	0.03	float	最高
lr	0.01	0.001	float	高
epochs	200	200	int	中
hidden1	128	128	int	中
hidden2	64	64	int	中
labeled_ratio	0.2	0.2	float	— 固定
use_adaptive_radius	True	True	bool	— 消融用
use_gcn	True	True	bool	— 消融用
dropout	0.5	0.5	float	— 硬编码
weight_decay	5×10^{-4}	5×10^{-4}	float	— 硬编码

NMIGOD 的参数设计体现了”以数据驱动为主，以人工调参为辅”的理念。其核心创新——基于连通分量信息熵的自适应半径 $\varepsilon_a = \sigma_a/(1 + \rho_a)$ ——使得算法能够自动感知每个属性的数据分布密度并动态调整邻域分辨率，大幅降低了对人工调节 λ 的依赖。在实践中，仅需调节 λ 和 τ 两个核心超参数即可覆盖大多数应用场景。